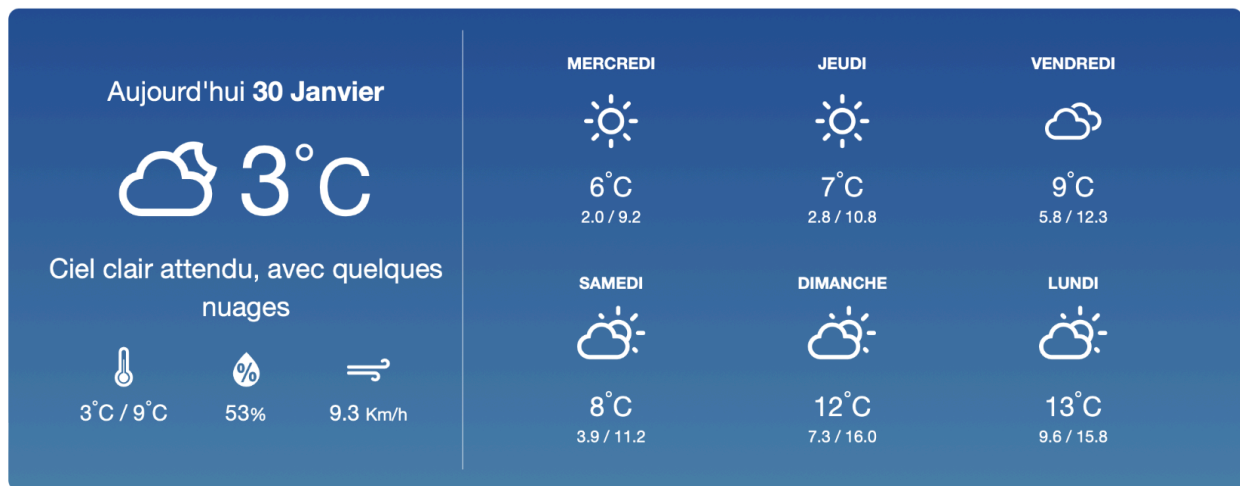


Project - Cloud and Advanced Analytics

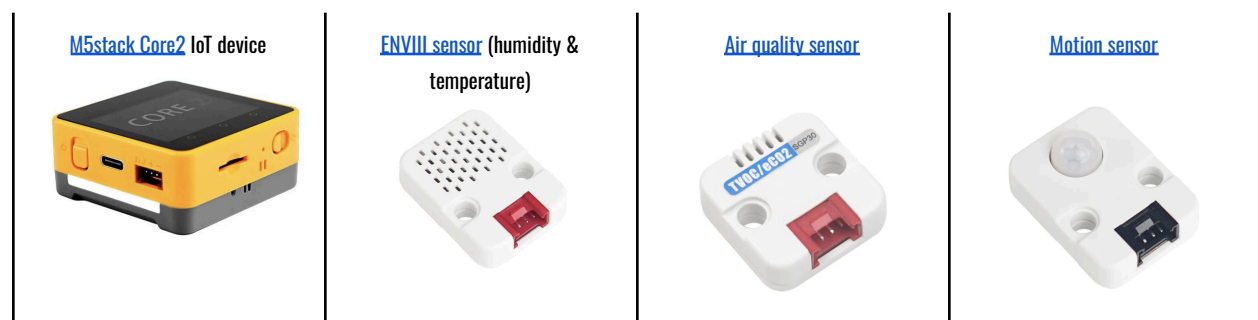
Project Update (2026): As Generative AI tools become widely accessible, we expect all submissions to reflect a seamless integration of these technologies underpinned by deep human oversight. Projects that fail to demonstrate exceptional polish will be interpreted as indicative of insufficient effort and a lack of rigorous engagement with the material.

Crucially, during your project presentation's Q&A session, you must demonstrate full mastery and understanding of every line of code submitted. Inability to do so will be viewed not just as a failure to meet academic standards, but also as a fundamental misunderstanding and breach of responsible AI usage.



Weather affects our mood, our clothing, our interactions with people ... even the stock market!

In this project, you will implement an indoor/outdoor weather monitor with a web dashboard using [M5stack](#) IoT devices and sensors. We will supply you with these devices and components, which must all be integrated in your solution (take care of them and return them in good, working condition).



You will gather both indoor measurements from the sensors and outdoor weather data from your location (a variable) using existing APIs and you will deploy this functionality on the cloud. Your interface on the M5stack IoT device should display:

- a) outdoor temperature
- b) indoor temperature and humidity
- c) indoor air quality
- d) Time and date (using [NTP](#))
- e) Weather forecast for upcoming days (*only this doesn't need to be stored in the cloud*)

You will also use the presence sensor to detect human activity and announce updates or critical information (e.g., very low air-humidity, storm coming up, take an umbrella, etc). All collected data will be stored in Google Cloud's BigQuery.

In addition to the interface on the device, you are required to develop a cloud-based dashboard using [streamlit](#), so that you monitor the conditions of your house from a distance. This user interface/dashboard will retrieve the data already sent to BigQuery. Therefore, you will have two interfaces, one on the M5stack (home) device and one on the cloud. These can be different, since on the streamlit (cloud) interface you mostly care about accessing not only the current data but also historical data.

Your solution is expected to use the following services and have these functionalities:

1. Big query for storing both the inside temperature/humidity, and the historical weather information. Issue alerts on the local dashboard (M5stack) if the indoor humidity falls below 40% or if the indoor air quality is bad.
2. For outside weather you will use the [openweathermap](#) API. You should also use icons for the various weather conditions. Check in GitHub for various projects using Python and openweathermap, so that you don't have to implement this part from scratch.
3. **Speech2text and text2speech** using APIs from Google or OpenAI. You can ask the device several questions, and make hooks to your historical or current data , eg "what was the temperature at home yesterday?", "Did humidity exceed 50% 2 days ago"? Alternatively, when presence is detected, the device will announce the weather, but only once every hour, for example, not constantly! (we don't want to annoy the user). In the morning if it is going to rain, remind the user to take an umbrella, etc. Make sure you create several such actions.
4. The UI (both on device and on the cloud) should display in a simple format the historical indoor temp, humidity, air quality, etc. **Note: we expect both interfaces to have a polished and professional look (icons, graphics, images, fonts). Be proud of what you make!**
5. It is important to consider that the device may be turned off. Upon turning on, the device should sync the last values from BigQuery, and the device should display those values. This is something that we will test in the evaluation of the project, so make sure you have some values in your BigQuery table! (and make sure the user can **change the WIFI!**).

User Interface

To understand how to make a usable and easy UI/UX experience especially on the small screen estate of the IoT device, examine existing weather monitors. Analyze how they present the information. Below are a few examples to draw inspiration from, but ultimately, you must devise your own interface.

You can also consult the Teaching Assistants for the course “Interaction Design”. They are aware of this and they are waiting eagerly to help you!



Code Quality - IMPORTANT!

For all projects, the focus should be on clean and reusable code (GenAI can help with this, BUT do this only after you have something working, rather than having generated code that you don't know how it works). In general, imagine that someone else other than you has to deploy your project. We expect a 3-tier architecture:

- data (bigquery)
- middleware/services (adding, retrieving data and logic)
- UI: a) Web UI Dashboard (streamlit), b) on-Device UI (Python)

These should/can be developed independently by the team members after the communication protocols, i.e., APIs, between them have been established.

Code length expectation:

- In the past, the total code for the project was expected to be **at least 1000 lines** (i.e., >500 per person). This includes the middleware in Flask/Python, and the streamlit visual dashboard. Today, since a lot of the code is AI-generated, we expect all submitted projects to be exceeding the 1000 lines.

> Evaluation criteria for code and program stability

- Is it clear how to deploy the application? Do you describe that in the [README.md](#) of your git?
- Have you abstracted the project variables (e.g., Wifi username/pass, local zip code, google cloud end-points) so that if someone wants to redeploy the application and services it should be easy?
- If everything is on the same file and not decomposed into meaningful packages, points will be deducted.
- If the device is disconnected from the internet or switched off, what happens when you turn the device on? This is something that you should also consider and we will test it during your presentation.

- Is your program stable? Have you tested all the corner cases?
- **IMPORTANT: You will present your project during our last class. Therefore, make sure that you have built in your UI the capability to change the WIFI credentials!** (use the “iot-unil” for your in-class presentation). Make sure you test this beforehand.

DELIVERABLES:

1. **VIDEO** (duration 3-10mins) in which you showcase the application: how to use it, which services you used etc. Video = unlisted video on YouTube. You shouldn't show any code! (that's what your git is for! ;) Just your solution. Imagine that this is something that you will post on LinkedIn. It should be exciting and highlight the benefits of your solution. In the end of the video, you can also give the link to your GitHub url so that people can see your code and deploy it. **The voice-over for the video should be your own, not AI, and you must show the team faces in the video in some form, e.g., in the beginning when introducing the video.**
2. **Github url:** Include your YouTube video link in your git. The README of the git should provide a general overview of the project and describe what each folder of the git contains. **CRITICAL: DON'T** include sensitive passwords in the Git repo or in the code!! (these should be environment variables or another file that is not included in the git). Ask your TAs or gpt how to do that.

On the top of your git, clearly declare the names of the group participants and **exactly what each person did for the project**. This should more or less be reflected in the git commits that we see for your project.

3. Submit a zip of your git in the **Moodle** submission.

For your video you can use slides if you like, but most important is to show your solution. For the project presentation during the last week, no slides are required, just to showcase all the features of your solution.

Live Presentation (last week of lectures):

This should be viewed as an examination rather than as a presentation, since a big part of your grade depends on that. So really come prepared! We will ask questions about your project and you have to show that you fully understand what you have submitted.

EVALUATION CRITERIA FOR FINAL GRADE:

The actual allotment is subject to change but at least this gives you an idea what we will evaluate.

Features & Code quality(40%)

- Google cloud URL for the running dashboard application (if this is not working no points will be given!)
- Google BigQuery
- Google Text-2-speech and Speech-2-text
- Google LLM/openAI (for generating realistic answer variations)
- Clean-code and abstracted variables. Different modules/functionalities should be abstracted in different files, functions or classes.
- Error handling in case of reset or loss of Internet.
- OpenWeatherMap or other weather service
- Weather forecast for the week, in addition to the current day.
- Appealing 3D printed enclosure for the M5stack and sensors [bonus points!]
- Innovative use of some aspects [bonus points!]. If you need more sensors from the M5stack ecosystem let us know! (e.g., cameras, fingerprint sensors, LORA, etc)

Quality of Interface (20%)

- User Interface. Should be appealing, intuitive and easy to use. Should not be slow.
- Do you include proper images, icons, fonts, color usage?

Quality of Git (20%)

- Well-structured: what the project does, images, link to video, etc
- Explains how to deploy the app? Is it easy to redeploy the app?

Quality of Video (20%) (Expected duration of 3-10mins)

- Is it captivating, exciting, fun? Is the recording audio of good quality?
- Do you have your own voice-over? Do you show the faces of the team?

Note: For those that do their own project we expect at least the following elements:

- Google Cloud, dashboard to view historical data and insights
- M5stack and sensors
- Github
- Video

And remember, it's the journey that matters, not the destination, so have fun while coding and deploying!

Some cool projects the students did with IoT devices the previous years:

- [Project time keeping device](#)
- [Language Translator](#)
- [Alcohol-level detector using CO2 sensor](#)
- [GPS Logbook for Rowing Club](#)
- [Smart Dog-food dispenser](#)

— END OF PROJECT DESCRIPTION —